



JAVA FULL-STACK · ENGINEERING PROJECT BRIEF

Project KEYSTONE

Field Service Management Platform

Client: Meridian Facilities Management · Your role: Full-Stack Engineer
4-Week Build · Spring Boot · React · PostgreSQL

CLIENT Meridian Facilities Mgmt	DOMAIN Java Full-Stack Engineering	DURATION 4 Weeks (1 Month)
DELIVERABLE Field Service Platform	ROLE Full-Stack Engineer	PROJECT WEIGHT 50 / 100 Marks

Document Control

Field	Detail
Document title	Project KEYSTONE — Field Service Management Platform
Document type	Engineering Project Brief & Technical Specification
Version	1.0
Status	Released
Client	Meridian Facilities Management
Delivery team	Zidio Development — Java Full-Stack Practice
Stack	Spring Boot 3 (Java 21) · React + TypeScript · PostgreSQL
Classification	Confidential

How to use this brief

Sections 01–08 define the product, the domain, and the architecture you are building. Section 09 specifies each feature with its acceptance criteria — use it as your build checklist. Sections 13–15 cover milestones, how the work is assessed, and what you submit.

Read Sections 05 (Domain Model), 07 (Work-Order Lifecycle), and 08 (Security) before writing any code — they define the data, the rules, and the access model the whole platform depends on.

Contents

Document Control.....	2
01 Project Overview.....	4
02 Client Background & Objectives	5
03 User Roles & Access	5
04 Scope & Deliverables.....	6
05 Domain Model	7
06 System Architecture.....	8
07 Work-Order Lifecycle	9
08 Security & Authentication	10
09 Feature Specification	11
10 API Design	13
11 Technology Stack	14
12 Build Plan — 4-Week Timeline.....	14
13 Milestones & Acceptance.....	16
14 Evaluation & Scoring.....	17
15 Submission Requirements	18
16 Engineering Standards & Ways of Working	19
17 Non-Functional Requirements.....	19
18 Risks, Assumptions & Constraints.....	20
19 Originality & Professional Conduct	20
20 Appendix A — Domain Glossary	21
21 Appendix B — Sample API Reference	21
22 Appendix C — Resources.....	22

01 Project Overview

KEYSTONE is the field-service platform Meridian Facilities Management uses to run its maintenance operation — from the moment a customer reports a problem to the moment the job is closed and billed. You are building it end to end: a Spring Boot back end, a React front end, and a PostgreSQL database behind them.

Meridian sends technicians to commercial buildings to service HVAC, electrical, and plumbing systems. Today that work is coordinated with spreadsheets, phone calls, and messaging apps. Jobs get lost, technicians arrive without the right information, and managers cannot see what is overdue. KEYSTONE replaces that with one system: dispatchers raise and assign work orders, technicians update them from the field, managers watch SLAs and dashboards, and customers can log requests and track progress.



Figure 1. KEYSTONE takes every inbound request through to a closed, accounted-for job.

1.1 What you are building

A single platform with four kinds of user — dispatcher, technician, manager, and customer — each seeing only what their role needs. Behind it sits a clean layered Spring Boot service with a governed work-order lifecycle, role-based security, parts and time tracking, SLA monitoring, and reporting. The full feature list and acceptance criteria are in Section 09.

What good looks like

A dispatcher can raise a work order, assign it to the right technician, and see it move across a board.

A technician can open their jobs on a phone, start work, log parts and time, and mark it complete.

A manager can see what is overdue, who is overloaded, and whether the team is hitting its SLAs.

A customer can raise a request and watch its status without phoning anyone.

02 Client Background & Objectives

2.1 The client

Meridian Facilities Management	
Business	Commercial facilities maintenance — HVAC, electrical, and plumbing services.
Operation	A team of field technicians serving multiple customers across many building sites.
Today	Work is coordinated by spreadsheet and messaging; no single system of record.
Pain	Lost jobs, poor visibility, missed response-time commitments, and disputes over what was done.

2.2 Objectives

1. Give Meridian one system of record for every work order, from request to close-out.
2. Let dispatchers assign and track jobs, and technicians update them from the field in real time.
3. Enforce a clear work-order lifecycle so jobs cannot skip steps or be quietly dropped.
4. Track response and resolution against SLAs, and surface breaches before they happen.
5. Give managers a live dashboard and give customers self-service visibility of their requests.

2.3 What success means

Outcome	How it shows up in the product
Nothing falls through	Every request becomes a tracked work order with an owner and a status.
Faster, fairer dispatch	Dispatchers assign by skill and load; technicians get full job context.
Accountable work	Parts, time, and a status history are recorded against every job.
Visible performance	Managers see overdue work and SLA compliance at a glance.
Less phone traffic	Customers self-serve requests and status instead of calling in.

03 User Roles & Access

KEYSTONE has four roles. Access is enforced on the server for every request — the UI hiding a button is never the security boundary. Role checks live in the service/controller layer.

Role	What they can do
Dispatcher	Create customers, sites, and work orders; assign work orders to technicians; see all jobs and the board.
Technician	See work orders assigned to them; start, hold, and complete jobs; log parts and time; upload photos.
Manager / Admin	Everything a dispatcher can do, plus close jobs, manage users and parts, and view all reports.

Role	What they can do
Customer	Raise requests for their own sites and view the status and history of their own work orders only.

3.1 Access rules

- Every endpoint is authenticated; anonymous access is limited to login and customer request submission where allowed.
- A customer can only ever see data for their own organisation's sites and work orders — never another customer's.
- A technician can only act on work orders assigned to them; they cannot reassign or close jobs.
- Destructive and administrative actions (user management, parts, close-out) are restricted to Manager / Admin.

Authorisation is server-side, always

Hiding a control in the React app is a usability choice, not a security control. Every protected action must be re-checked on the server with Spring Security (method-level `@PreAuthorize` or equivalent). Assume a determined user will call your API directly.

04 Scope & Deliverables

Scope is fixed so the platform is finishable and reviewable in four weeks. Deliver everything in scope to a professional standard before adding anything beyond it.

4.1 Deliverables

#	Deliverable	What it is
D1	Back-end service	A layered Spring Boot REST API with security, validation, and migrations.
D2	Database & schema	A PostgreSQL schema managed by versioned Flyway migrations and seed data.
D3	Web front end	A React + TypeScript SPA covering all four roles.
D4	Work-order lifecycle	The governed state machine with a full status-history audit trail.
D5	Dashboard & reporting	Operational views: status counts, overdue work, SLA compliance.
D6	API documentation	OpenAPI/Swagger docs for the whole API.
D7	Deployment	Back end, front end, and database deployed and reachable, with seed logins.

4.2 In scope

- Customer, site, and work-order management with the full lifecycle and audit trail.
- Role-based access for dispatcher, technician, manager, and customer.
- Parts usage, time logging, SLA tracking, a dashboard, and a customer portal.

4.3 Out of scope

Explicitly excluded — building these instead of the required scope will lower your score.

- Payment processing, invoicing engines, and accounting integrations (track parts/time only).
- Native mobile apps (the technician view is a responsive web page).
- Route optimisation, GPS tracking, and automated scheduling algorithms.
- Third-party integrations with real facilities or ERP systems.

Definition of done

A deliverable is done only when it meets its acceptance criteria in Section 09, is covered by the security and validation rules, runs from a clean checkout following the README, and is deployed. “It compiles locally” is not done.

05 Domain Model

The domain is small but real. A customer has sites; sites raise work orders; work orders are assigned to technicians and accumulate a status history, parts used, and time logged. Model these as JPA entities with the relationships shown below. Full column detail is in Appendix A.

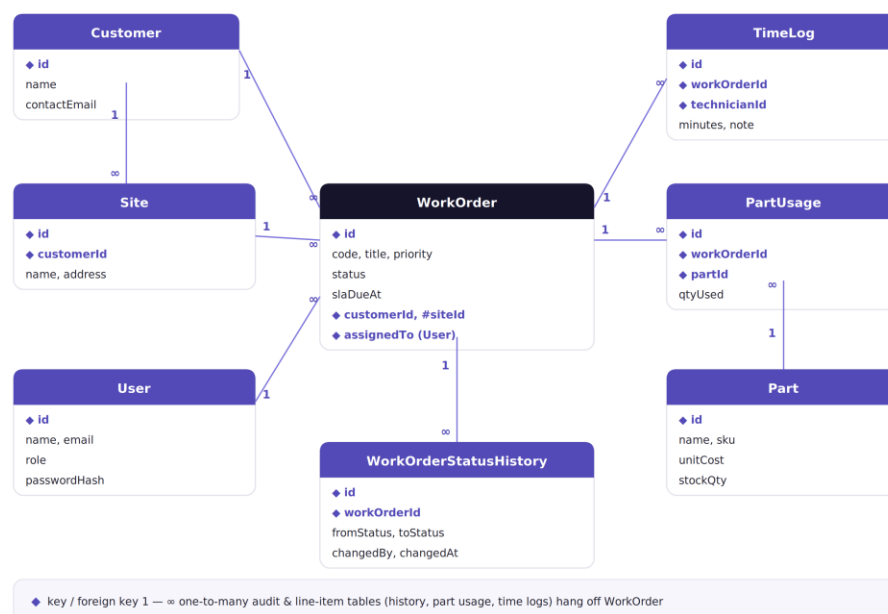


Figure 2. Core domain model — *WorkOrder* is the heart of the system; audit and line-item tables hang off it.

5.1 Entities

Entity	Purpose
Customer	An organisation Meridian serves; owns one or more sites.
Site	A building/location belonging to a customer; where work happens.

Entity	Purpose
WorkOrder	A unit of work: title, priority, status, SLA due date, customer, site, assignee.
User	A platform user with a role (dispatcher, technician, manager, customer).
WorkOrderStatusHistory	An append-only audit row written on every status change.
Part / PartUsage	Inventory items and the quantities consumed on a work order.
TimeLog	Labour logged by a technician against a work order.

Integrity rules

A work order must always belong to a customer and a site. Status changes are append-only in the history table (never edited or deleted). Using parts on a job and decrementing stock must happen in one transaction — partial updates are not allowed.

06 System Architecture

KEYSTONE is a classic layered Spring Boot application behind a React SPA. Requests flow from the browser to controllers, into services that hold the business rules, down to JPA repositories and PostgreSQL. Keep the layers clean: controllers stay thin, business logic lives in services, and entities never leak directly to the client — map to DTOs.

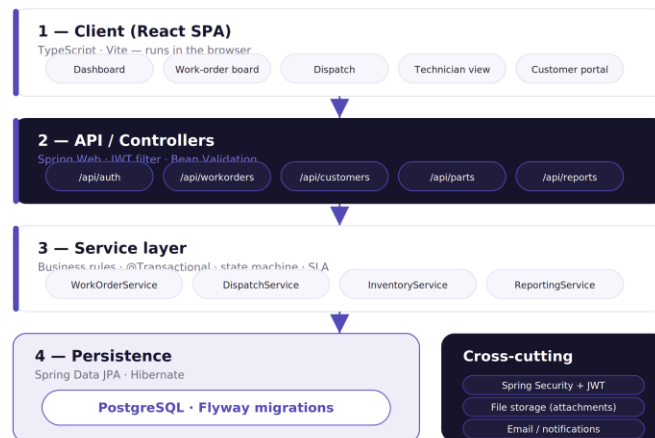


Figure 3. System architecture — a React SPA over a layered Spring Boot service and a PostgreSQL database.

6.1 Layer responsibilities

Layer	Responsibility
Controllers	HTTP only: validate input, enforce auth, map DTOs, delegate to services. No business logic.
Services	Business rules, the work-order state machine, SLA logic; transactional boundaries (@Transactional).

Layer	Responsibility
Repositories	Spring Data JPA data access; queries scoped so customers only read their own data.
Persistence	PostgreSQL with Flyway-managed migrations; no schema changes by hand.

6.2 Principles

- Thin controllers, rich services: business decisions live in one place and are testable.
- DTOs at the boundary: never serialise JPA entities straight to the client.
- Transactions guard invariants: multi-step changes (status + history + stock) commit together or not at all.
- Migrations, not magic: every schema change is a reviewed Flyway script, runnable on a clean database.

07 Work-Order Lifecycle

The work-order lifecycle is the core business rule of the platform. A work order moves through a fixed set of states, and only certain transitions are allowed. Enforce this in the service layer — not just in the UI — and write an audit row on every change.

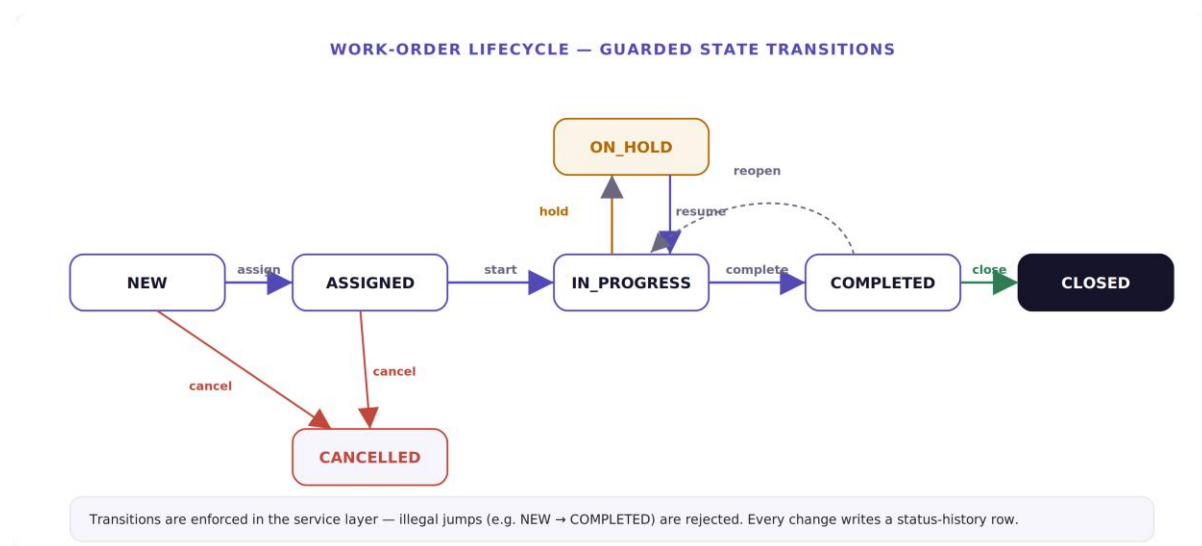


Figure 4. The governed work-order lifecycle. Illegal transitions are rejected; every change is recorded.

7.1 States

State	Meaning
NEW	Raised but not yet assigned to a technician.
ASSIGNED	Allocated to a technician, not yet started.
IN_PROGRESS	The technician is actively working the job.
ON_HOLD	Paused (e.g. waiting on parts or access).

State	Meaning
COMPLETED	Work finished, awaiting manager / customer sign-off.
CLOSED	Signed off and accounted for — terminal.
CANCELLED	Abandoned before completion — terminal.

7.2 Rules

- Only the transitions shown in the diagram are permitted; any other change is rejected with a clear error.
- Each transition is restricted by role (e.g. only a manager can CLOSE; only the assigned technician can start).
- Every transition writes a WorkOrderStatusHistory row: from, to, who, when, and an optional note.
- Terminal states (CLOSED, CANCELLED) cannot transition further.

08 Security & Authentication

Security is a first-class requirement, not an afterthought. KEYSTONE uses stateless JWT authentication with Spring Security: users log in once, receive a signed token, and present it on every request. The server holds no session.

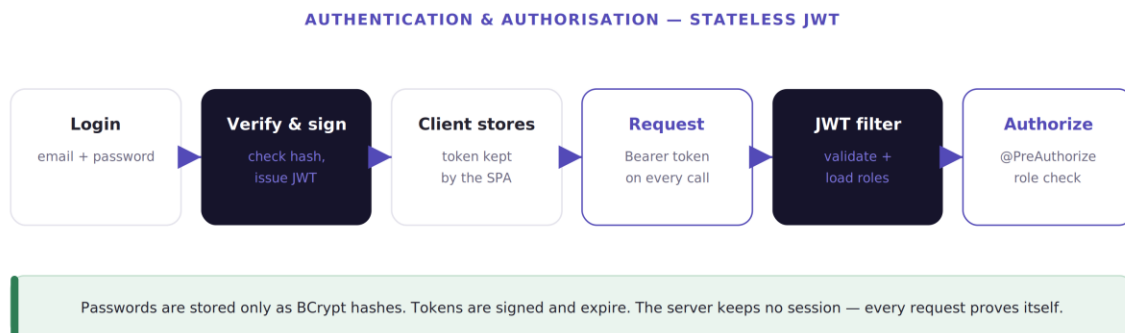


Figure 5. Stateless authentication: log in, receive a signed token, and prove identity and role on every request.

8.1 Requirements

- Passwords are stored only as BCrypt hashes — never in plain text or reversible form.
- JWTs are signed with a server secret and expire; expired or tampered tokens are rejected.
- Every protected endpoint enforces role with Spring Security (method-level authorisation).
- Sensitive values (DB credentials, JWT secret) come from configuration/environment, never hard-coded or committed.
- Input is validated on the server (Bean Validation); errors return a consistent, structured response.

Threats to defend against

Assume the API is called directly, not just through your UI. Defend against: a customer requesting another customer's work orders by changing an ID; a technician closing a job they are not allowed to; and any request without a valid token reaching protected data. Test these cases.

09 Feature Specification

Each feature below has acceptance criteria — the definition of done used in review. Treat them as your checklist.

9.1 Core platform

F1 Authentication & roles

Acceptance criteria

1. Users log in and receive a JWT.
2. Four roles exist with distinct permissions.
3. Protected endpoints enforce role on the server.
4. Passwords are BCrypt-hashed; tokens expire.

F2 Customers & sites

Acceptance criteria

1. Dispatchers/managers can create and edit customers and their sites.
2. A site always belongs to a customer.
3. List views are searchable and paginated.
4. Customers see only their own organisation's data.

F3 Work-order management

Acceptance criteria

1. Create a work order with title, description, priority, customer, and site.
2. Server-side validation on all inputs.
3. Editable while open; immutable once closed/cancelled.
4. Each work order has a unique human-readable code.

F4 Dispatch & assignment

Acceptance criteria

1. A dispatcher can assign a work order to a technician.
2. Assignment moves the order to ASSIGNED and notifies the technician.
3. Reassignment is allowed while the job is open.
4. A board (Kanban by status) shows all open work.

F5 Technician field view

Acceptance criteria

1. A technician sees only their assigned jobs.
2. They can start, hold/resume, and complete a job.
3. They can log parts used and time spent.
4. The view is responsive and usable on a phone.

F6 Parts & time logging

Acceptance criteria

1. Logging parts on a job decrements stock in one transaction.
2. Time entries record minutes and an optional note.
3. Totals (parts cost, labour time) roll up on the work order.
4. Stock cannot go negative.

F7 SLA tracking & notifications

Acceptance criteria

1. Each work order gets an SLA due date based on priority.
2. A scheduled job flags work orders at risk of, or in, breach.
3. Breaches are visible to managers and trigger a notification.
4. SLA status is shown on the board and dashboard.

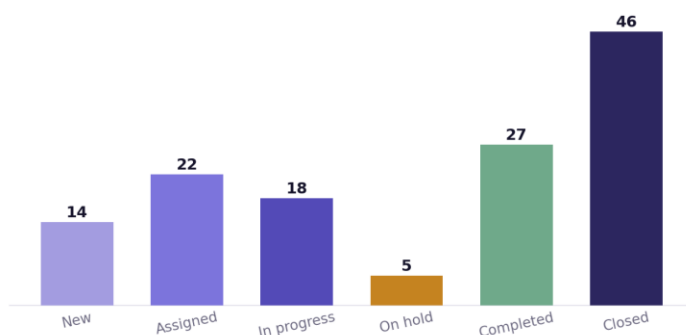
F8 Dashboard & reporting

Acceptance criteria

1. Managers see counts by status, overdue work, and SLA compliance.
2. Figures reflect the current data and any active filters.
3. At least one view breaks results down by technician or site.
4. Empty and loading states are handled.

Illustrative operations dashboard — the kind of view managers open each morning

Work orders by status



SLA compliance (30 days)

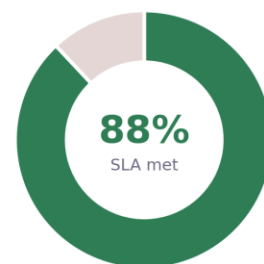


Figure 6. The kind of operational view F8 should produce: status mix and SLA compliance at a glance.

F9 Customer portal

Acceptance criteria

1. A customer can raise a request for one of their sites.
2. They can view status and history of their own work orders only.
3. They cannot see other customers' data or internal fields.
4. Requests enter the same work-order pipeline.

10 API Design

Design the REST API to be predictable and conventional. Use nouns for resources, proper HTTP verbs and status codes, consistent error responses, and pagination on list endpoints. A representative slice is below; the full reference is in Appendix B.

Endpoint	Method	Purpose
/api/auth/login	POST	Authenticate; returns a JWT.
/api/work-orders	GET	List work orders (filterable, paginated, role-scoped).
/api/work-orders	POST	Create a work order.
/api/work-orders/{id}	GET	Fetch one work order with its history.
/api/work-orders/{id}/assign	POST	Assign to a technician.
/api/work-orders/{id}/status	POST	Transition status (validated against the lifecycle).
/api/work-orders/{id}/parts	POST	Log parts used (transactional).
/api/reports/summary	GET	Dashboard metrics.

10.1 Conventions

- Status codes mean what they say: 200/201 success, 400 validation, 401/403 auth, 404 missing, 409 illegal transition.
- Errors return a consistent JSON shape (timestamp, status, message, field errors) — never a stack trace.
- List endpoints support pagination, sorting, and filtering; never return unbounded result sets.
- The whole API is documented with OpenAPI/Swagger and browsable.

11 Technology Stack

Layer	Technology	Why
Language	Java 21	Modern LTS; records, pattern matching.
Back-end framework	Spring Boot 3 (Web, Validation)	Industry-standard layered services.
Security	Spring Security + JWT	Stateless auth and role enforcement.

Layer	Technology	Why
Persistence	Spring Data JPA / Hibernate	Mapping and repositories.
Database	PostgreSQL	Reliable relational store.
Migrations	Flyway	Versioned, reviewable schema changes.
Front end	React + TypeScript (Vite)	Typed, component-based SPA.
API docs	springdoc-openapi (Swagger UI)	Live, browsable API reference.
Build / deploy	Maven or Gradle · Docker · cloud host	Reproducible builds and deployment.

Allowed alternatives

Angular instead of React, and Gradle instead of Maven, are acceptable if you are stronger there — confirm with your mentor. The back-end stack (Spring Boot, Spring Security, JPA, PostgreSQL, Flyway) is fixed, because the engineering practices it enforces are the point of this project.

12 Build Plan — 4-Week Timeline

The build runs four weeks (twenty working days). Each week ends in a reviewable increment. Work the weeks in order — the foundation, then core work orders, then the rules that make it real, then productionising.

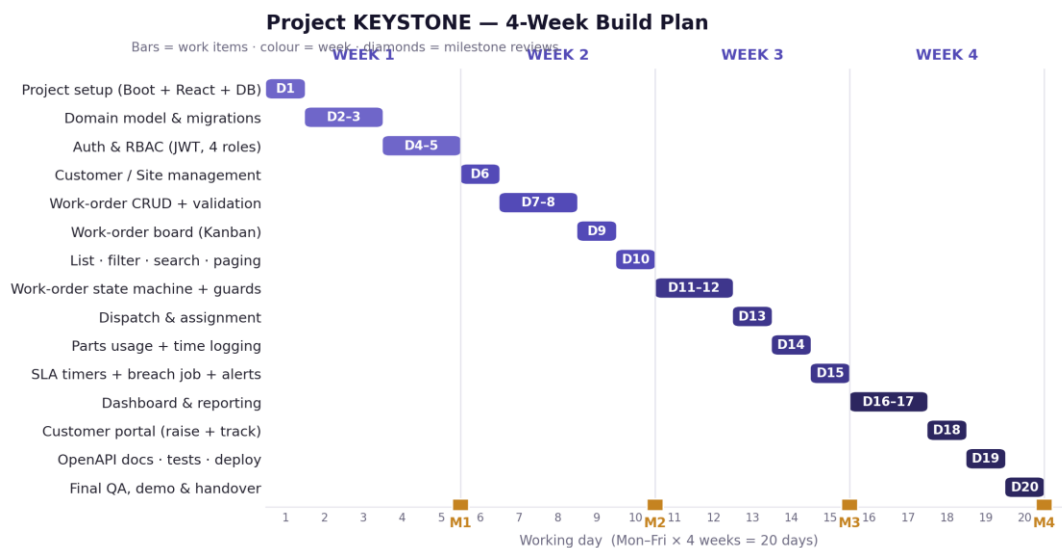


Figure 7. The four-week build plan, with milestone reviews at the end of each week (M1–M4).

WEEK 1	Foundation End-of-week increment: Running skeleton: auth, domain model, migrations, seed data.
-----------	---

Day 1	Set up the Spring Boot project, React app, PostgreSQL, and Flyway; get a hello-world deploy working.
Day 2–3	Model the domain entities and write the first migrations; seed reference data.
Day 4–5	Build authentication and RBAC (JWT, four roles). Review M1.

WEEK

2

Core Work Orders**End-of-week increment:** Work orders end to end: create, list, board, with validation.

Day 6	Customer and site management (API + UI).
Day 7–8	Work-order CRUD with server-side validation and DTOs.
Day 9	The work-order board (Kanban by status) in the SPA.
Day 10	List, filter, search, and pagination. Review M2.

WEEK

3

Workflow & Rules**End-of-week increment:** The lifecycle, dispatch, parts/time, and SLA tracking.

Day 11–12	Implement the work-order state machine with guarded transitions and history.
Day 13	Dispatch and assignment, plus the technician field view.
Day 14	Parts usage and time logging (transactional; stock integrity).
Day 15	SLA due dates, the scheduled breach check, and notifications. Review M3.

WEEK

4

Productize**End-of-week increment:** Dashboard, customer portal, API docs, tests, and deployment.

Day 16–17	Dashboard and reporting (status counts, overdue, SLA compliance).
Day 18	Customer portal and technician-view polish (attachments).
Day 19	OpenAPI docs, key tests, and full deployment.
Day 20	Final QA against acceptance criteria; record the demo. Review M4.

If you fall behind

Protect the work-order lifecycle, security, and deployment (Weeks 3–4) — they carry the most marks and define a real platform. Trim dashboard polish or the breadth of reporting before you trim the core rules. Tell your mentor early if you are blocked.

13 Milestones & Acceptance

Four milestones gate the build, each ending in a review of a working increment. Hitting them on time is part of delivering like a professional engineer.

Milestone	End of	Increment	Project pts	Status
M1 — Foundation	Week 1	Auth, RBAC, domain model, migrations, seed.	10	☐
M2 — Core work orders	Week 2	Work-order CRUD, board, list/filter/search.	10	☐
M3 — Workflow & rules	Week 3	Lifecycle, dispatch, parts/time, SLA.	20	☐
M4 — Productize	Week 4	Dashboard, customer portal, docs, deploy.	10	☐

Why M3 carries the most marks

The lifecycle, security, transactional integrity, and SLA logic are what make KEYSTONE a real platform rather than a CRUD form. They are the hardest part and the most revealing of engineering skill. Spend your best effort in Week 3.

13.1 Optional stretch goals

Only after the full scope is excellent. These strengthen a strong submission; they never substitute for core work.

- Email or in-app notifications on assignment and SLA breach.
- Technician availability/skills so dispatch can suggest the best match.
- Audit log viewer in the UI built from the status-history data.
- A small integration-test suite covering the lifecycle and authorisation rules.

14 Evaluation & Scoring

The project is scored within Zidio's 100-point rubric. This brief governs the Project (50) and Submission (20) components; Attendance (20) and Test (10) are assessed separately.

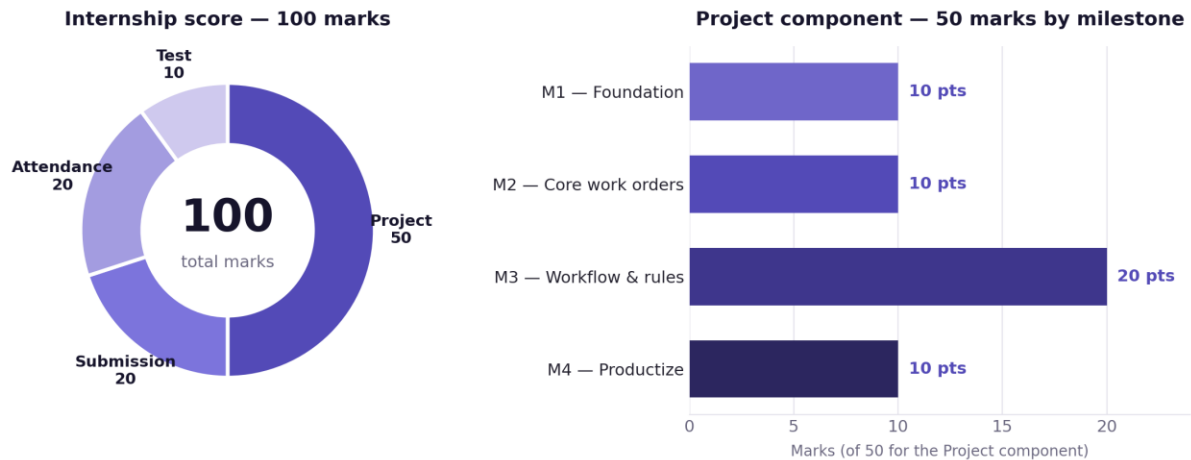


Figure 8. Mark distribution across the 100-point rubric, and the 50-point project broken down by milestone.

14.1 Project breakdown (50 marks)

Milestone	Marks	What earns full marks
M1 Foundation	10	Clean layered structure, working JWT/RBAC, sound entities, and migrations that run on a fresh DB.
M2 Core work orders	10	Validated CRUD, a usable board, and proper list/filter/search/pagination.
M3 Workflow & rules	20	A correctly enforced lifecycle, server-side authorisation, transactional parts/time, and working SLA logic.
M4 Productize	10	A useful dashboard, a working customer portal, API docs, and a live deployment.

14.2 Submission breakdown (20 marks)

Criterion	Marks	What earns full marks
Code quality & structure	5	Clean layering, DTOs, error handling, meaningful commits, and a clear README.
Deployment	5	Back end, front end, and database live and reachable with seed logins.
API documentation	5	Complete, accurate OpenAPI/Swagger covering the endpoints.
Demo video	5	A 3–5 minute walkthrough covering each role and the lifecycle end to end.

Quality bar — what separates a top score

Average submission: work orders that can be created and listed, but a lifecycle enforced only in the UI, authorisation gaps, and no deployment.

Top submission: a deployed platform where the lifecycle and access rules hold even against direct API calls, parts/time are transactionally correct, and a manager could actually run their day from the dashboard.

15 Submission Requirements

Submit all of the following by the deadline. A missing item is a missing mark.

1. Git repository (public, or private with mentor access) for the back end and front end.
2. Live URLs: the deployed API, the deployed front end, and Swagger UI.
3. Seed login credentials for each of the four roles.
4. A README with: overview, stack, local setup, environment variables, how to run migrations/seed, and an architecture summary.
5. A 3–5 minute demo video (unlisted link) walking through each role and the work-order lifecycle.
6. The completed submission form with all links.

Clean-checkout rule

A reviewer should be able to clone your repository, follow the README, run the migrations and seed, and have the platform working. If it only runs on your machine, treat that as a defect to fix before submitting.

16 Engineering Standards & Ways of Working

16.1 Standards

- Layered structure: controllers thin, services hold logic, repositories for data; entities never serialised to clients.
- Validate all input on the server; handle errors centrally (@ControllerAdvice) with a consistent response shape.
- No secrets in the repo: configuration via environment; .env and credentials are git-ignored.
- Meaningful commits and small pull-requests; the main branch always builds.
- Tests where they matter most: the lifecycle transitions and the authorisation rules.

16.2 Cadence

Ritual	Purpose
Daily async update	What you did, what's next, any blockers.
Weekly milestone review	Demo the working increment to your mentor; get direction. Mandatory.
Mentor office hours	Technical blockers and code review.

16.3 Suggested repository structure

```
keystone/  
  backend/                # Spring Boot service  
    src/main/java/.../controller  # thin REST controllers  
    src/main/java/.../service    # business logic + state machine  
    src/main/java/.../repository  # Spring Data JPA
```

```

src/main/java/.../domain      # entities
src/main/java/.../security    # JWT, filters, config
src/main/resources/db/migration # Flyway scripts
frontend/                    # React + TypeScript (Vite)
docker-compose.yml
README.md

```

17 Non-Functional Requirements

Quality	Requirement
Security	Stateless JWT; BCrypt passwords; server-side role checks on every protected action.
Data integrity	Multi-step changes are transactional; the status history is append-only; stock never goes negative.
Validation	All inputs validated server-side; consistent, structured error responses.
Performance	List endpoints paginated; no unbounded queries; common views respond quickly on seed-scale data.
Usability	The technician view works on a phone; the dashboard answers a manager's key questions at a glance.
Observability	Sensible logging of key actions (assignment, status change, breach) for support and audit.
Reproducibility	Clean checkout + README + migrations brings the platform up from nothing.

18 Risks, Assumptions & Constraints

18.1 Assumptions

- Technicians have a phone with a browser; no native app is required.
- Notifications can be email or in-app; no SMS/telephony integration is required.
- Data volumes are moderate (seed-scale); no big-data or sharding concerns.

18.2 Risks & mitigations

Risk	Mitigation
Lifecycle enforced only in the UI	Put transition rules in the service layer; test illegal jumps are rejected.
Authorisation gaps	Re-check role on every protected endpoint; test cross-customer access is blocked.
Scope creep into invoicing/scheduling	Hold the line on scope; track parts/time only; no optimisation engines.
Deployment left to the last day	Deploy a skeleton in Week 1 and keep it current; never big-bang at the end.

18.3 Constraints

- The back-end stack (Spring Boot, Spring Security, JPA, PostgreSQL, Flyway) is fixed.
- The work must be deployed, not only runnable locally.
- Delivery window is four weeks; scope is fixed to fit it.

19 Originality & Professional Conduct

- You may use AI assistants, libraries, and documentation — that is normal practice. You must understand and be able to explain every part of what you submit.
- You may follow tutorials, but the implementation of this platform must be your own work.
- Never commit secrets, and never copy another trainee's repository.
- At the final review, mentors may ask you to explain or modify any part of the code live — including the lifecycle and the security layer.

20 Appendix A — Domain Glossary

Term	Meaning
Work order	A unit of maintenance work: what needs doing, where, for whom, by when.
Dispatcher	The person who raises and assigns work orders to technicians.
Technician	The field engineer who performs the work and updates the job.
Site	A specific building/location belonging to a customer.
Asset	A piece of equipment at a site (e.g. an HVAC unit) that work relates to.
SLA	Service-Level Agreement — the response/resolution time the job must meet.
Status history	The append-only record of every state change on a work order.
Lead time	Used for stock: time between ordering a part and it being available.

21 Appendix B — Sample API Reference

Endpoint	Method	Notes
/api/auth/login	POST	Body: email, password. Returns JWT + role.
/api/customers	GET / POST	List or create customers (manager/dispatcher).
/api/customers/{id}/sites	GET / POST	Sites for a customer.
/api/work-orders	GET / POST	List (role-scoped, paged) or create.
/api/work-orders/{id}	GET / PUT	Fetch (with history) or edit while open.
/api/work-orders/{id}/assign	POST	Assign to a technician (dispatcher/manager).

Endpoint	Method	Notes
/api/work-orders/{id}/status	POST	Transition status; 409 if illegal.
/api/work-orders/{id}/parts	POST	Log parts; decrements stock transactionally.
/api/work-orders/{id}/time	POST	Log time against the job.
/api/reports/summary	GET	Dashboard metrics (manager).

This is indicative, not exhaustive — design the full surface consistently and document it with OpenAPI.

22 Appendix C — Resources

Topic	Link
Spring Boot	https://docs.spring.io/spring-boot/index.html
Spring Security	https://docs.spring.io/spring-security/reference
Spring Data JPA	https://docs.spring.io/spring-data/jpa/reference
Flyway	https://documentation.red-gate.com/flyway
PostgreSQL	https://www.postgresql.org/docs
React	https://react.dev
springdoc-openapi	https://springdoc.org